

Programming Assignment 1 Report: Fast Fourier Transform

Subject: 41052 Advanced Algorithms

Name: Felix Herrmann

Student ID: 25483227

|Table of contents

Table of contents-----	2
(a) What I built-----	3
(b) Empirical study-----	4
(c) What I learned-----	8
(d) AI use-----	9

| (a) What I built

Algorithm

I implemented the fast fourier transform (FFT), an algorithm designed to compute a fourier transform in $O(n \log n)$ time. I also implemented the predecessor to FFT, the discrete fourier transform (DFT) in order to conduct an empirical study comparing the performance of these two algorithms.

The FFT uses the principle of divide and conquer, dividing the inputs into odd and even arrays repeatedly until reaching the base case of 1. The problem is then conquered by recombining the divided sections after modifying the odd half with a “twiddle” factor (I labelled it as rotation instead). This gives us the frequency distribution of the input.

All of the algorithms assume that the input is 1 second including the input generators. 1hz will always have 1 cycle no matter the input size, 2hz will always have 2 cycles, etc.

Language

I built the algorithm in C++ due to my familiarity with it and its extensive usage in this course.

Data representation

I represented almost all arrays in the code with `std::vector<double>` or `std::vector<std::vector<double>>`. This made it easier to convert data types for use in different functions and provided suitable accuracy for the algorithm. I created functions for a variety of small tasks like averaging arrays, printing arrays and generating inputs, and implemented a 1D and 2D version when necessary.

Over/under delivery

I did not implement the inverse of any of the functions. The main goal of this report is to investigate the performance of FFT and comparable algorithms, and the inverse functions have the same performance so I did not bother to implement/test them. The implementation of FFT works only on inputs 2^n as is normal in the base implementation. The discrete cosine transform was also implemented.

Navigating the code

There is one file, `main.cpp`, that contains all the code for the FFT and testing of the FFT. `README` tells you how to compile and run the program. Within `main.cpp` there are some pre-made test inputs that demonstrate functions that are not used in the final program, but could still be useful.

(b) Empirical study

What I compared

The runtime of DFT and FFT at different input sizes.

Hypotheses

The FFT will scale much better than the DFT but may have worse performance at very small input sizes (<8).

Generation of data

I generated data by running the program in test mode which runs the algorithm 5 times, timing it each time. The times are then output in the terminal.

```
PS C:\Users\felix\Desktop\uni\Advanced Algorithms\FFT> .\FFT.exe test dft 10
FFT!
Input length: 1024
1: 4345.98
1: 4345.98
1: 4345.98
1: 4345.98
1: 4345.98
Times in us
101321,95420,103084,97120,97967,
PS C:\Users\felix\Desktop\uni\Advanced Algorithms\FFT> 
```

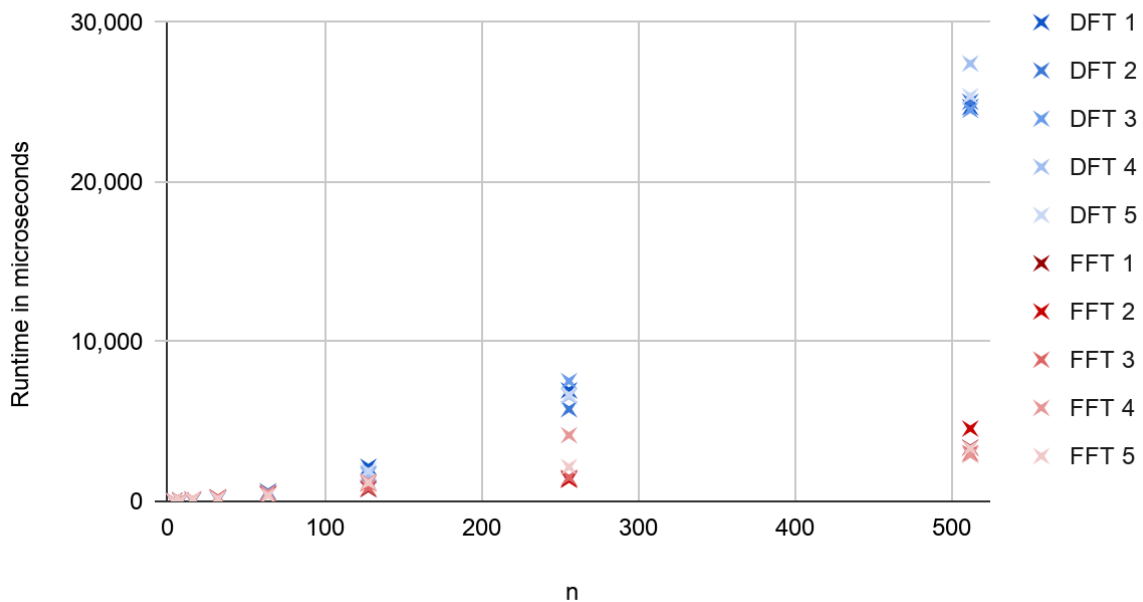
I manually did this for DFT and FFT for all the input sizes I wanted and copied the outputs into google sheets for further analysis.

n	Algorithm runtime in microseconds												
	DFT							FFT					
	DFT 1	DFT 2	DFT 3	DFT 4	DFT 5	DFT avg		FFT 1	FFT 2	FFT 3	FFT 4	FFT 5	FFT avg
4	6	13	7	6	7	7.80		30	13	12	10	21	17.20
8	28	15	30	15	60	29.60		61	36	32	55	32	43.20
16	36	36	35	40	37	36.80		95	121	88	103	119	105.20
32	112	156	147	141	139	139.00		219	236	209	175	180	203.80
64	611	425	450	414	423	464.80		428	451	481	554	361	455.00
128	2160	1724	1629	1684	1907	1820.80		758	1129	829	1248	1165	1025.80
256	6946	5768	7531	6578	6667	6698.00		1398	1352	1486	4142	2154	2106.40
512	24692	25022	24507	27419	25377	25403.40		3364	4556	3023	2917	3324	3436.80
1024	97548	115269	110010	103516	105272	106323.00		8991	6023	6948	5872	6965	6959.80
2048	373868	373072	371847	383306	374764	375371.40		13624	13912	14137	16134	14312	14423.80

Measurement

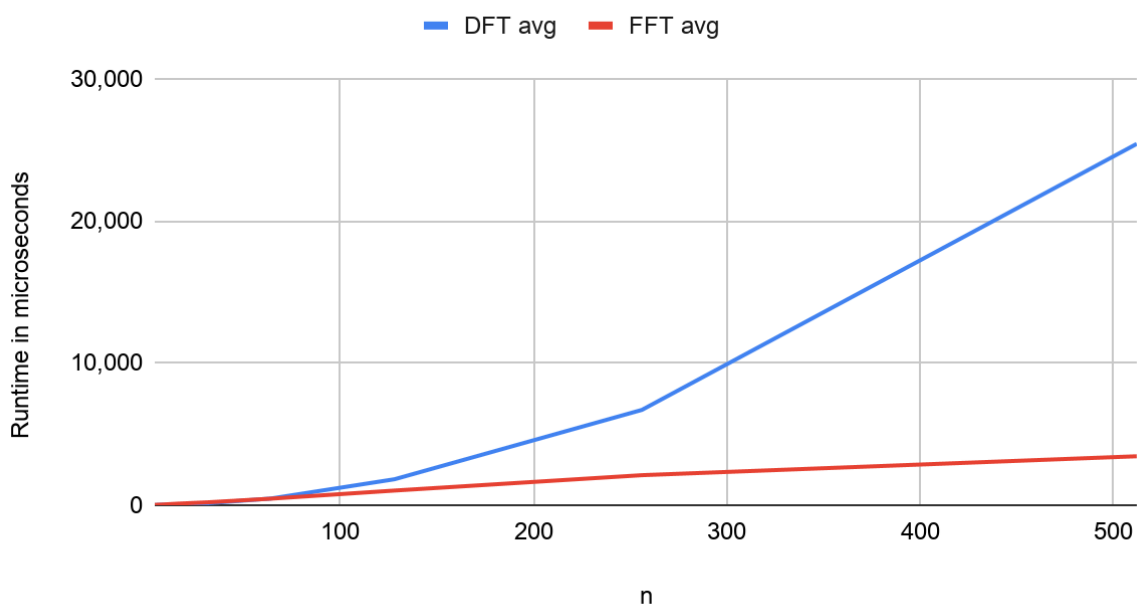
I created multiple graphs from these values: one scatter plot of all the DFT and FFT test values, one line graph of the average DFT and FFT values, one line graph of the average FFT values, one line graph of the average DFT/FFT.

Algorithm runtime in microseconds vs n



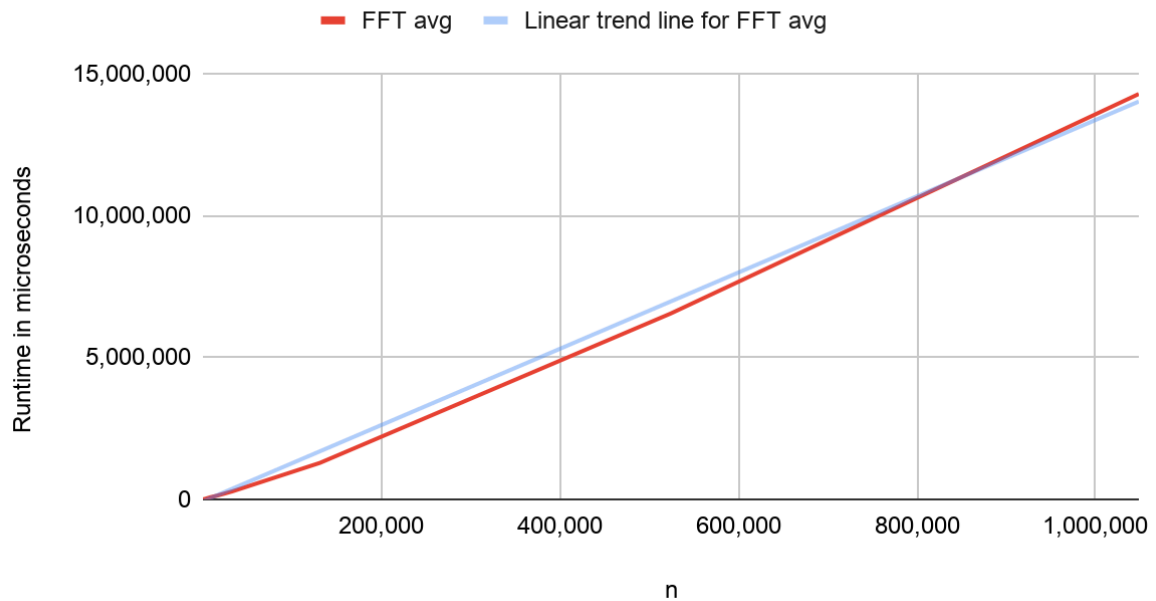
This shows that DFT runtime is exponential, the FFT runtime is too compressed to tell how its runtime is changing.

Average algorithm runtime in microseconds vs n



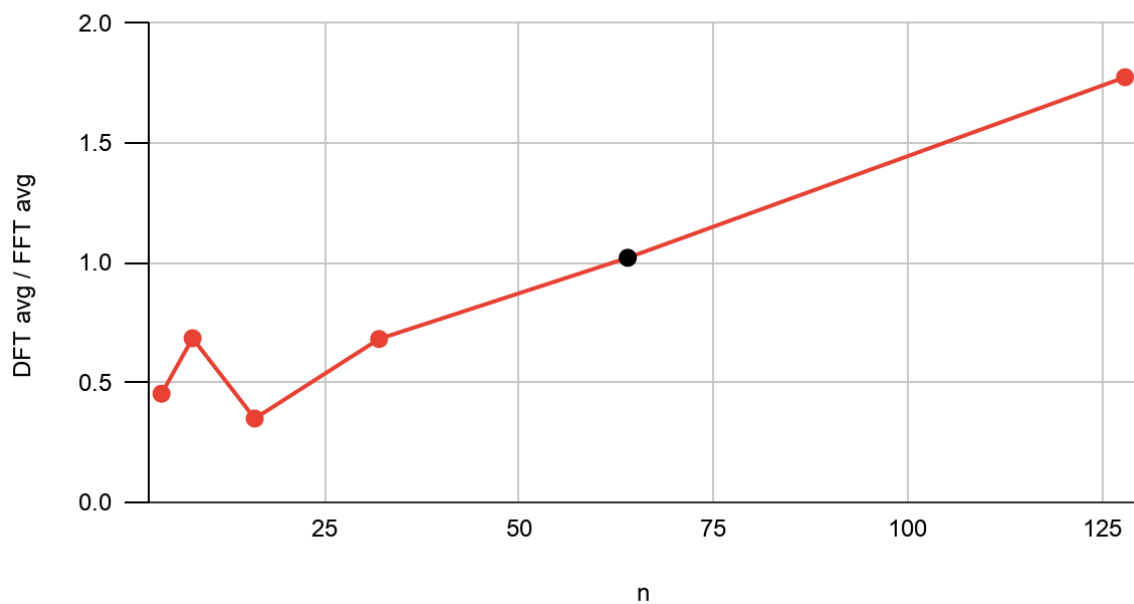
The same observations of DFT being exponential can be made from the average runtime. The average runtime graph shows that FFT is faster for large values.

FFT Average algorithm runtime in microseconds vs n



It is difficult to see by eye any $O(n \log n)$ relation, appearing almost linear. A linear trend line does not fit the data showing that it is not a linear relation.

DFT avg / FFT avg vs n



This graph shows that DFT has a faster runtime than FFT for small values and slower one for larger values. DFT and FFT have a very similar runtime at $n=64$, with a ratio of 1.021.

Analysis

The FFT does scale better than the DFT and also performs worse at very small input sizes. The range in which DFT performs better was larger than expected, performing evenly to FFT at $n=64$ and faster than FFT at $n<64$. I believe the worse performance at small input sizes is due to the overhead of the FFT algorithm, I didn't expect the overhead to be so large and I believe this is because I underestimated the overhead as only recursion and didn't account for more arithmetic operations.

The FFT had time complexity $O(n \log n)$ and the DFT had time complexity $O(n^2)$, both as expected.

|(c) What I learned

What surprised me

I was surprised by the overhead of the FFT, I estimated that it would perform worse for input sizes of <8 , but it was much more at <64 .

I was surprised at how easy the implementation of the FFT was compared to the theory side of it.

I was surprised by how large the difference in runtime is on realistic real world input sizes. At 32,000 inputs, FFT ran over 400x faster than DFT.

What did I get wrong

I initially implemented discrete cosine transform (DCT) instead of DFT, this ignores complex values entirely. When I reached the point where I had to implement FFT I realised this mistake and implemented DFT. I also implemented DFT in an incredibly slow manner, saving all the “wrapped” inputs and then summing them afterwards. I fixed this by adding each “wrapped” input to a variable as it was calculated, preventing the need to go through the array again. I had many off by one errors or incorrect usage of sin vs cos or not negating the output of the aforementioned.

What did I learn

I learnt how incredibly important FFT is and how much faster $O(n \log n)$ is than $O(n^2)$ when run on realistic input sizes like $n = 32768$ (almost the same amount of samples in 1 second of 44.1kHz audio). Using DFT takes on average ~2 minutes compared to $\sim 1/4$ of a second for FFT. Such a massive difference on a realistic size, real world input drives home how important FFT is.

I also learnt that the overhead of a divide and conquer algorithm is more complicated than just the overhead for recursion. Divide and conquer algorithms can require more time consuming operations that negate the improved time complexity at low input sizes.

|(d) AI use

I in general, do not use any generative AI tools, and did not use any generative AI tools in the creation of this report. I understand that this section is a large part of the marks for this assessment so I have filled out the answers as best as I can in the context of not using generative AI tools.

Tools

I used VS code for implementation of all code. I used Google Docs for writing the report. I used Google Sheets for compiling results and creating graphs. I used Git mainly to be able to access the project from multiple devices. I used Github to host my Git repository. I used Desmos to help me understand the complicated mechanics of the fourier transform and to build models that I could refer back to when implementing the DFT. I used OBS to record my video.

What

All of the code was written in VS code taking about 16 hours all up, as well as the README. I used Desmos extensively at the start to help me understand fourier transforms and to make a basic mathematical implementation of DFT before beginning implementation. I ended up using Desmos for probably 2-3 hours. I used Google Docs and Sheets for about 3-4 hours in the writing of the report and collation of results.

Examples of incorrectness

I am almost certain that my implementation of DFT does not allow for an inverse DFT, and I believe it is happening because my calculation of the imaginary portion of a point when wrapped is incorrect. I noticed this very late in development and I believe it was working at some point and I must have broken it. The real values for my DFT and FFT also don't match but both do return the correct result and are capable of finding multiple frequencies combined.

Human vs AI workload

Whilst I coded everything I do not believe I fully understand how the integration of twiddle factor leads to the average positions of a "wrapped" input. In future I would investigate how each step of recursion contributes to the final average position.

I do not fully understand why the -1 in $(2 * \text{speed}) - 1$ of the DCT and DFT is necessary but I am fairly certain the result is correct.

PS

If you have any more questions in regards to this section please let me know, I have done my best to answer these questions in good faith and am happy to re-answer any sections if my interpretation in these circumstances did not match yours.